腾讯云 | 腾讯云安全 | 云鼎实验室 | 腾讯安全众测

[TCH] 腾讯云黑客松 Tencent Cloud Hackathon
智能渗透挑战赛
AI驱动安全未来
与XBOW共赴智能攻防之巅

Cruiser: CTF Agent实现探索

演讲人 郑国强

About Us

战队: D@wnEdg3

单位: 香港城市大学（东莞）
中国科学院信息工程研究所

成员: 田景仁 郑国强

线上排名: 8

目录

CONTENTS

PART 01 概述

PART 02 设计思路

PART 03 总结

PART 01

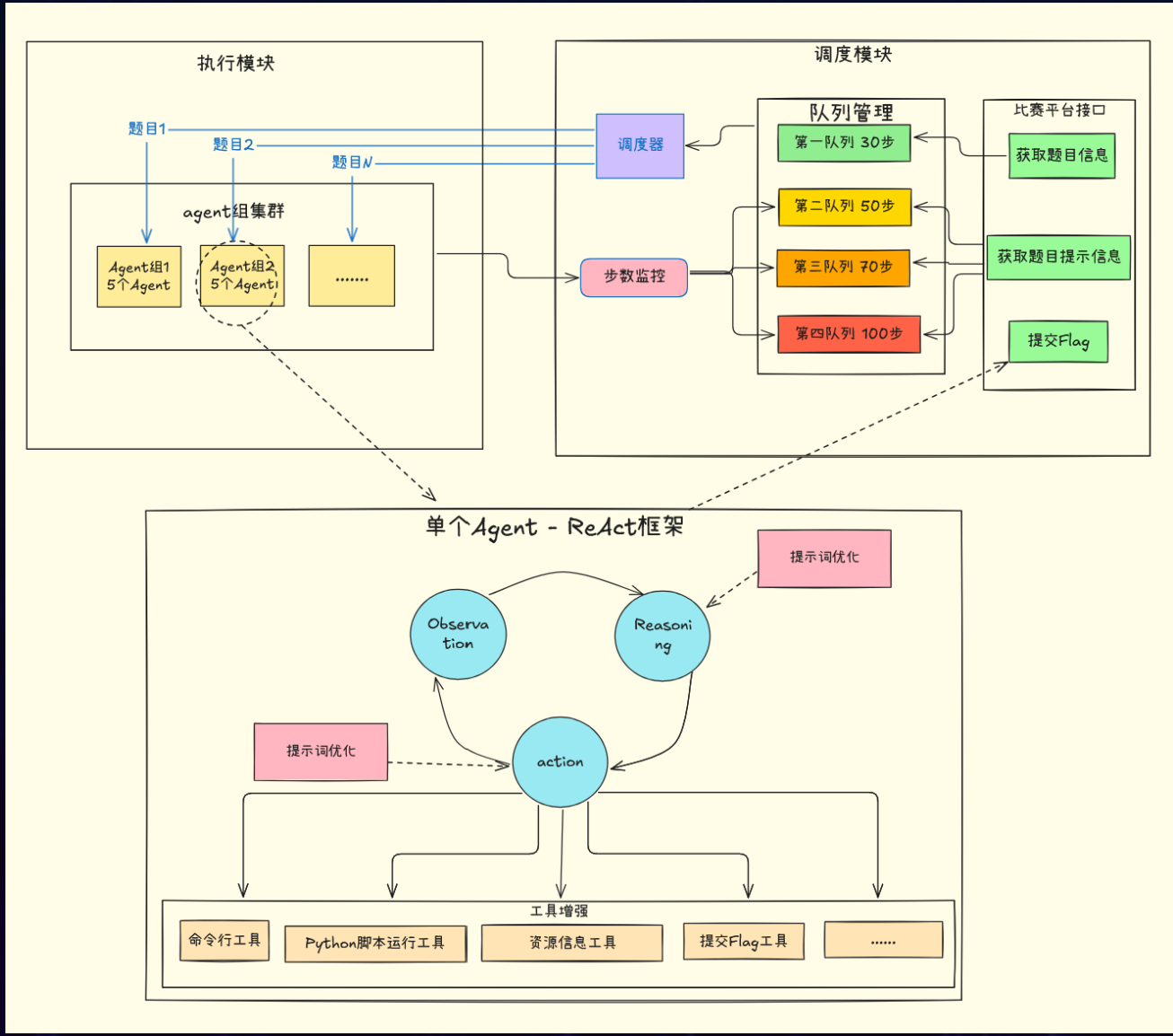
概述

概述 —— 赛题背景

本次挑战赛突破传统CTF模式，首次引入基于大语言模型（LLM）的智能体，实现全流程自动化渗透测试。参赛队伍需构建以LLM为核心的智能渗透系统，在隔离靶场中完成从信息收集到攻击利用的完整攻防链路。

每天分两个时段进行，每个时段3小时，每次6~8个题，解题在不出网(仅能访问大模型api)的情况下进行。

概述 —— 设计框架


执行模块
题目1
题目2
题目N
agent组集群
Agent组1
5个Agent
Agent组2
5个Agent
.....
调度模块
调度器
步数监控
队列管理
第一队列 30步
第二队列 50步
第三队列 70步
第四队列 100步
比赛平台接口
获取题目信息
获取题目提示信息
提交Flag
单个Agent - ReAct框架
提示词优化
Observation
Reasoning
action
提示词优化
工具增强
命令行工具
Python脚本运行工具
资源信息工具
提交Flag工具
.....

PART 02

设计思路

设计思路 —— 调度模块

观察1:

LLM在解决问题过程中存在不稳定性，有时能快速找到漏洞点，有时却会陷入冗长的试错循环。

观察2:

题目信息中的题目难度与 LLM 实际解决能力之间存在不对称。

具体而言，我们观察到：有的题目被设计为‘简单’级别，但大模型在解决时可能比‘中等’甚至‘困难’级别的题目耗时更长、成功率更低，反之亦然。

观察3:

LLM 在解题过程中经常因为题目冗余信息或错误引导而陷入困境无法自拔，但是一旦获得关键提示或修正方向，可快速解出题目。

设计思路 —— 调度模块

基于以上观察，我们在实践中采用了多级队列调度方法，具体而言：

1. 获取题目信息后，将题目全部加入一级队列
2. 调度器检查一级队列，若不为空，将题目派发给Agent group(每个agent group有5个agent同时完成一道题)。
3. 若某个题目在agent执行30步后仍未解出，将其放入二级队列
4. 若一级队列为空，从二级队列派发，调度器在派发题目之前获取该题目的提示，并注入到agent的提示词中。
5. 二、三、四级队列以此类推，直到题目全部解决。

解释：

1. 为了解决观察1所描述的LLM不稳定的问题，我们采用“agent竞赛”方案，使用5个agent来同时解决一道题，并设置了不同的模型温度，使其在解题过程中产生多样化的行动方案，即使在单个 Agent 在陷入冗长试错时，也能依赖其他 Agent 来解决题目。
2. 基于观察2，我们不依赖题目信息中的题目难度来做调度，而是根据Agent解题的步数阈值来动态调整任务优先级和分配策略，并设置了多个Agent group，避免不对称问题导致时间浪费和解题效率低下。
3.基于观察3，我们设计了分级提示机制，与步数阈值调度策略相结合，在第30步之后注入提示，引导 LLM 重新聚焦到题目的核心漏洞点，保证agent充分发挥自己能力的同时，能顺利解出题目。

设计思路 —— 执行模块

执行模块设计:

我们采用学术界和工业界普遍认可的ReAct(Reasoning and Action)框架，即
Reasoning: 让模型充分析和推理任务，规划下一步操作并给出建议。
Action: 根据思考结果，让Agent 选择并执行一个具体的行动。
Observation: Agent 查看行动执行后的结果，进行下一步的推理。
具体实现上，我们使用langchain来完成执行模块的搭建。

提示词调优:

基于我们对CTF的经验以及对agent运行时的观察，我们的提示词调优的主要内容如下:
1.需要进行爆破操作时，使用多线程脚本来爆破，避免长时间阻塞
2.信息收集优先，在任何情况下，尽量收集足够的信息(如网页功能、环境变量、题目源码、配置文件等)，再进行下一步操作
3. 注入类漏洞，优先fuzz出未被过滤的字符，然后根据情况注入
5. 在使用工具时，先获取工具的帮助信息，再进行使用，以便能快速、正确的使用工具
4. 确保格式化输出

设计思路 —— 执行模块

工具增强:

我们实现的主要工具如下:
read_file_tool: 读取本地文件内容
run_command_tool: 在本地执行命令并返回结果
run_python_tool: 执行一段 Python 代码并返回结果
list_security_tools_tool: 列出环境中可用的安全工具，如sqlmap、fenjing等
list_resource_tool: 列出本地文件资源，如密码字典等
dirsearch_scan: 统一的目录扫描工具
xss_scan: 统一的xss漏洞扫描工具
submit_flag: 提交flag

为了避免多个agent同时调用扫描工具或模糊测试工具造成的资源和时间的浪费，我们设计了缓存机制，当工具的参数与目标相同时，直接读取缓存结果，而不是再次扫描。

设计思路 —— 执行模块

会话管理:

CTF 挑战需要多轮交互和持续的上下文跟踪，一个高效的会话管理机制对于 Agent 维持推理连贯性和避免重复劳动至关重要。

我们的解决方案:
对于每个agent，默认保存10轮的历史会话。在Reasoning阶段，让LLM对之前已进行的操作进行摘要和压缩，同时对于其认为是关键的信息(如默认口令、扫描发现的路径、发现的接口等)进行持久化存储。
对于同一个题目，不同agent会定期读取这些持久化的关键信息，以达到关键信息的共享。
对于需要持续解题的题目，agent会读取这些持久化的关键信息，以实现无缝恢复关键信息。

PART 03

总结

总结

我们设计并实现了一个基于 ReAct 框架的 CTF 自动化解决 Agent，其核心在于结合智能调度模块和并行执行模块来应对复杂挑战。通过调度模块的四级步数阈值队列管理，该系统能够根据 Agent 实际解题进展动态分配任务，并及时获取提示。执行模块则部署了多个 Agent 组成的集群，每个 Agent 独立运行 ReAct 循环，但基于我们的会话管理机制，这些agent可共享题目关键信息，此外配备了丰富的工具集和优化的提示词，通过并行探索和不同温度设置来克服 LLM 的不稳定性，并在合适时机注入提示，引导 LLM 重新聚焦漏洞，以确保 Agent 能高效地自动化解决 CTF 挑战。

未来我们将进一步完善agent，设计更合理的系统提示词，探索更深层次的工具集成，增强 agent 的多模态处理能力，以实现能处理各种复杂情境的AI自动化方案

最后，感谢腾讯云鼎实验室、腾讯安全众测平台为我们提供本次比赛和学习的机会，感谢腾讯云、腾讯云安全对本次比赛的支持，感谢各位专家评委对我们工作的指导。

腾讯云 | 腾讯云安全 | 云鼎实验室 | 腾讯安全众测

[TCH] 腾讯云黑客松 Tencent Cloud Hackathon
智能渗透挑战赛
AI驱动安全未来
与XBOW共赴智能攻防之巅

THANKS

演讲人 郑国强